# Trabalho Prático 1 - Múltiplas Ordenações

João Gabriel Guimarães Alves Vaz

Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2024

## 1 Introdução

O trabalho proposto tratava de um problema de ordenação de registro por múltiplas chaves. Foi entregue um arquivo contendo informações sobre pessoas (CPF, nome e endereço). O objetivo do trabalho era implementar um programa que fosse capaz de ordenar eficientemente esse grande registro com base em qualquer uma das 3 chaves mencionadas. Dessa forma, a situação-problema deixa evidente a necessidade de utilizar a ordenação indireta. Ou seja, o registro original foi ordenado utilizando apenas apontadores para as chaves - para que não fosse necessário mover, em memória, os massivos itens do registro diversas vezes.

Abaixo está um resumo do que será abordado em cada uma das sessões desse documento.

- Na sessão 1, há uma introdução ao tema, o contexto e o sumário;
- Na sessão 2, há o método de implementação;
- Na sessão 3, há a análise de complexidade dos procedimentos implementados;
- Na sessão 4, há uma discussão das estrategias de robustez utilizadas;
- Na sessão 5, há uma análise experimental;
- Na sessão 6, há a conclusão desta documentação;
- Na sessão 7, há a bibliografia.

## 2 Método

O programa foi desenvolvido na linguagem C++, compilada pelo compilador G++ da GNU Compiler Collection

### 2.1 Funções de ordenação

Para realizar a ordenação por nome, CPF e endereço pedidas, foram implementados 3 métodos de ordenação: o quicksort (com mediana de 3 e uso da inserção para subvetores pequenos), o mergesort e o insertionsort. Todos as 3 funções de ordenação implementadas podem ser utilizadas para ordenar quaisquer um dos 3 campos. Ou seja, os 3 algoritmos são capazes de ordenar por qualquer um dos 3 campos. Esse "agnosticismo" por parte das funções de ordenação foi feita utilizando um ponteiro para membro de struct. A função de ordenação recebe um ponteiro para o membro do struct Pessoa (CPF, nome ou endereço) a partir do qual a ordenação deve ser realizada. Essa escolha de implementação só foi possível tendo em vista que o trabalho foi implementado em C++ - uma vez que, em C, essa feature é inexistente.

### 2.2 Tipos abstratos de dados

Para a modularização do código, foi implementado um tipo abstrato de dados chamado Pessoa. Esse tipo, declarado como um struct, armazena os dados de CPF, nome, endereço e "outros" de cada Pessoa do arquivo xcsv. Cada linha desse arquivo tem uma instância de pessoa associada. Para a ordenação indireta, foi utilizado um apontador duplo (vetor de ponteiro); esses foram ordenados ao mudar o endereço para o qual o apontador aponta (dependendo do resultado das comparações feitas nas funções de ordenação). Ou seja, foram inicializados 3 arrays de ponteiros, um para CPF, um para nome e um para endereço. Inicialmente, a correspondência é direta: o i-esimo elemento de cada array aponta para a i-esima pessoa do vetor dinâmico Pessoa*. Então, as funções de ordenação alteram essa correspondencia, comparando a chave de cada apontador e mudando para onde ele aponta - realizando, assim, a ordenação indireta.

### 2.3 Módulos

São três os principais módulos: main, Pessoa e utils. No primeiro, como o proprio nome diz, há a implementação da função main, que chama as demais funções conforme o código avança. No segundo, está a definição do tipo abstrato de dados Pessoa. Note que esse não possui um correspondente .cpp; ou seja, por ser um struct, não há um arquivo de implementação (já que não há métodos no struct Pessoa). Por último, o arquivo utils.cpp implementa utilitários; funções auxiliares. Funções de leitura de arquivo, de limpeza de memória e de ordenação foram implementadas nesse arquivo.

### 2.4 Instruções de compilação

Para executar o programa, basta chamar o utilitário *make* - que vai compilar o executável e colocá-lo em *./bin/tp*1.*out* - e chamar o programa, dessa forma:

```
make
./bin/tp1.out <entrada>
```

## 3 Análise de complexidade

### 3.1 Tempo

Como mencionado anteriormente, a função main é a responsável por chamar as demais funções. Portanto, em termos de análise assintótica, a complexidade da função é a complexidade do "módulo" (mais especificamente, do trecho de código) de maior complexidade dentre todos os trechos de código que compõe a função main. Sendo assim, a complexidade de tempo da função main é $O(n^2)$ - que é a complexidade da função insertionsort. Realizando uma análise mais aprofundada, a complexidade da função main é determinada pela complexidade de suas sub-funções:

- A função leArquivo, chamada no início da main, executa um laço de repetição que itera em cada linha do arquivo de entrada. Assim, a função tem complexidade $O(n)$.
- O laço de repetição que faz a correspondência inicial do array de ponteiros percorre a entrada, item por item. Logo, sua complexidade é $O(n)$.
- O quicksort e o mergesort tem complexidade $O(nlog(n))$ - como descrito nos slides da disciplina.
- O insertionsort é $O(n^2)$ - também como mencionado no recurso disponibilizado pelos docentes.
- A função chamaSort, que chama o algoritmo de ordenação, tem complexidade fixa de O(n) (por causa do for loop que percorre a entrada item a item), mais um custo variável que depende do algoritmo de ordenação que ela chama. No pior caso, o insertionsort é chamado, o que faz a complexidade variável do chamaSort ser $O(n^2)$. Portanto, a complexidade final da funcao é $O(n^2)$.
- O laço de repetição para a impressão da tabela ordenada percorre cada um dos $n$ elementos da entrada apenas uma vez; então, sua complexidade se torna $O(n)$.
- A função de limpeza da memória tem complexidade fixa, $O(1)$, pois apenas executa o comando delete nos vetores alocados dinamicamente.

Assim sendo, a complexidade temporal total do programa é a maior de suas complexidades parciais; ou seja, $O(n^2)$.

### 3.2 Espaço

Em termos de espaço de memória, a complexidade é linear no tamanho da entrada. Isso porque, para cada uma das $n$ pessoas, são alocadas:

- Uma instância do struct Pessoa
- Um item em 3 vetores de ponteiros

Além disso, tem-se ainda a memoria extra alocada pelo mergesort, que é linear no tamanho da entrada. Sendo assim, tem-se $O(4n) + O(kn)$ - o primeiro termo dos tipos abstratos de dados e dos apontadores utilizados; o segundo, do mergesort. Portanto, a complexidade espacial total do programa continua linear no tamanho da entrada: $O(4n) + O(kn) = O((4 + k)n) = O(n)$.

Em termos de complexidade por função, tem-se:

- main: Vetor de pessoas e vetor de ponteiros para endereços, CPFs e nomes: $O(4n)$ no total.
- leArquivo: Não aloca memória extra, $O(1)$.
- quicksort: Esse algoritmo de ordenação é in-place, portanto, $O(1)$.
- mergesort: Memória extra linear no tamanho da entrada; $O(n)$.
- insertionsort: Algoritmo in-place; $O(1)$.
- chamaSort: Apenas chama um dos 3 algoritmos de ordenação acima; $O(1)$.
- limpaMemoria: $O(1)$.

## 4 Estratégias de robustez

Ao longo do trabalho, foram implementadas em diversas partes do código, blocos de try-catch. Esses blocos de programação defensiva foram a principal estratégia de robustez utilizada para impedir falhas na execução do programa. Especificamente, já no início da função main há o primeiro desses blocos, responsável por garantir a abertura sem erros do arquivo xcsv de leitura da entrada. Um pouco abaixo desse ponto, há mais um de tais blocos, responsável por garantir que há memória disponível no heap para a alocação dos vetores dinâmicos. Por fim, há um último bloco que garante que a leitura do arquivo seja realizada corretamente. Esse serve para casos em que alguma das pessoas da entrada tenha informações faltando (ie, para o funcionamento correto do programa, todas as pessoas da entrada devem possuir CPF, nome, endereço e o campo outros com valores bem-definidos; nenhum desses campos pode assumir valor NULL ou vazio).

## 5 Análise experimental

Todos os testes abaixo detalhados foram feitos numa máquina com as seguintes especificações:

- Processador: Intel i7, 7th gen, 8 cores
- RAM: 16 GB
- Armazenamento: 256GB SSD, 1TB HD
- SO: Linux (Manjaro 24.0.4)

Para realizar a análise experimental, foram feitos alguns testes variando o tamanho da entrada e/ ou de atributos específicos da entrada (o número de caracteres da coluna "outros"), além de algumas propriedades dos dados de entrada do programa (dados ordenados, inversamente ordenados, ordem aleatória). Os resultados estão apresentados abaixo. A seguinte nomenclatura foi usada: mxn significa m linhas e o tamanho (em caracteres) da coluna "outros", n. O tempo em segundos foi obtido pela ferramenta time. Em todos os testes realizados, o Valgrind (ferramenta cachegrind) acusou misses dos cachês I1, LLi, D1, LLd, LL de 0.05%, 0.05%, 2.1%, 1.4%, 0.4%, respectivamente.

Apesar de que a ideia original era utilizar a ferramenta Memlog, desenvolvida pelo professor Meira, não consegui executar a ferramenta e obter resultados satisfatórios. Na falta de qualquer documentação ou de qualquer sucesso de meus pares, desisti completamente do uso dela e recorri ao Valgrind e Cachegrind para análise dos cache misses e da localidade de referência – como sugerido em uma das Práticas de Aprendizagem da disciplina.

## 5.1 Tempo

| **Tipo de Ordenação** | **Tamanho (Linhas x Payload)** | **Tempo (s)** |
|---|---|---|
| Aleatória | 1000 x 1000 | 0,094 |
| Aleatória | 1000 x 5000 | 0,141 |
| Aleatória | 5000 x 1000 | 0,877 |
| Aleatória | 5000 x 5000 | 0,934 |
| Nome (ordem direta) | 1000 x 1000 | 0,091 |
| Nome (ordem direta) | 1000 x 5000 | 0,107 |
| Nome (ordem direta) | 5000 x 1000 | 0,751 |
| Nome (ordem direta) | 5000 x 5000 | 0,812 |
| Nome (ordem inversa) | 1000 x 1000 | 0,090 |
| Nome (ordem inversa) | 1000 x 5000 | 0,115 |
| Nome (ordem inversa) | 5000 x 1000 | 0,725 |
| Nome (ordem inversa) | 5000 x 5000 | 0,863 |

Table 1: Tabela de tempos para diferentes tipos de ordenação

A partir desses resultados, pode-se perceber uma tendêcia interessante:

- O tamanho da entrada (m) é o fator predominante no tempo de execução. Ou seja, o tamanho da entrada importa mais do que o tamanho da coluna outros (n).
- Ainda assim, embora o tamanho da entrada seja mais importante, o tamanho da coluna outros não é insignificante. Em outras palavras, o tempo de execução é sim perceptivelmente impactado pelo tamanho da coluna "outros": experimentos x:1000 levam, em média, menos tempo que experimentos x:5000.
- A ordenação parcial da tabela, feita, nesse caso, com base na coluna "nome", também impacta o tempo de execução. Testes nos quais a tabela foi pré-ordenada pela coluna "nome" executam mais rápido que os testes em ordem aleatória. Além disso, testes nos quais a ordenação foi inversa executam mais devagar que os testes em ordem aleatória.
- Tratando da localidade de referência, como o enunciado previa, no geral, é ruim. Extamente pelo aspecto de buscar dados na memória, característico do problema, a localidade de referência "sofre".

## 5.2 Localidade de referência

Por fim, seguem alguns gráficos de localidade de referência. Com esses gráficos, pode-se perceber outras tendências curiosas:

- Locais de DP alta: momento da impressão, por causa da ordenação indireta característica do trabalho.

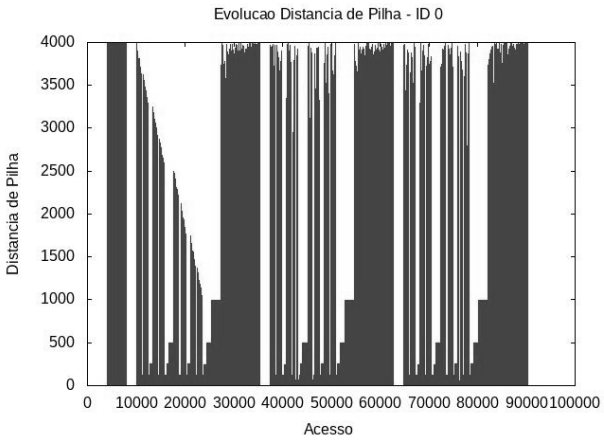



Figure 1: Execução do mergesort com ordem aleatória

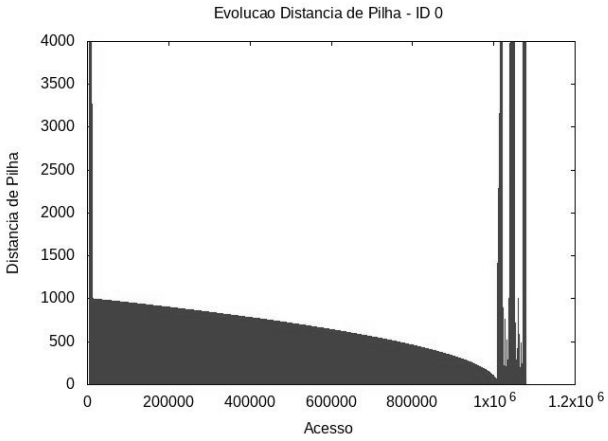



Figure 2: Execução do quicksort com ordem aleatória

- mergesort: a DP começa alta e vai reduzindo, para depois aumentar novamente. Isso é devido ao comportamento do algoritmo: a DP diminui pois o vetor está sendo ordenado; logo após isso, a outra metade do vetor começa a ser ordenada, aumentando a DP novamente.
- quicksort: a escolha do pivô e a partição favorecem a localidade espacial (elementos próximos são acessados), o que favorece a DP.

# 6 Conclusões

Em conclusão, o "trabalho prático 1: múltiplas ordenações" girou em torno da ordenação indireta múltipla: de uma tabela de pessoas, por CPF, nome e endereço. O trabalho foi muito útil no sentido de aumentar a prática com a linguagem C++ e aprofundar os conceitos de ordenação e de análise assintótica de algoritmos. Dentre os aspectos técnicos aprendidos, algo muito interessante foi o uso da referência para membro do struct, utilizado para tornar as funções de ordenação "agnósticas" do campo. Além disso, aspectos relacionados a modularização, programação defensiva, estratégias de robustez e eficiência de código também foram exercitados. Fora isso, a habilidade de documentar código de forma eficiente - ou seja, se atendo aos detalhes essenciais, sem ser muito verboso - também foi exercitada.

# 7 Bibliografia

Meira, W. (2024). Slides virtuais da disciplina de estruturas de dados. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.